

CASO PRÁCTICO 1 UD 7 Interfaces

José Cruces Aparicio 2º DAM

CREACIÓN DE RPM

Resuelve

Desde tu experiencia, sabes que muchas empresas y profesionales que llevan a cabo la realización de softwares para hardware específicos deben tener en cuenta el sistema operativo, por ejemplo, Linux.

Existen muchas empresas que realizan tales desarrollos con Java, llevando la instalación a cabo un técnico de manera manual. Sin embargo, muchas veces se opta por la automatización de este proceso empaquetando los desarrollos en RPMs que puedan ser instalados por usuarios de una forma más sencilla.

Realiza una búsqueda de información respecto al proceso de creación de un RPM. Tras esto, realiza el procedimiento incluyendo las herramientas necesarias y empaqueta una aplicación que tengas hecha en JAVA (JAR). Por último, realiza el procedimiento de la instalación una vez que el paquete RPM esté realizado.

Recursos

Se deberá consultar el contenido de la unidad, internet, libros, revistas y utilizar medios informáticos para la presentación de la tarea (Word, Power-Point...).

Objetivos

Conocer el procedimiento para empaquetar un programa para el sistema operativo Linux.

Saber empaquetar código Java en un fichero JAR.

CASO PRÁCTICO 1 - CREACIÓN DE RPM CON JAVA EN DOCKER

1. Introducción

En este documento se describe el proceso completo de creación de un paquete RPM a partir de una aplicación Java, utilizando un entorno Fedora ejecutado mediante Docker Desktop en macOS. Se detalla desde la preparación del entorno hasta la instalación final del paquete.

1.1 ¿Qué es un paquete RPM?

Un paquete RPM (Red Hat Package Manager) es un formato de distribución de software utilizado en sistemas Linux como Fedora, Red Hat o CentOS. Este sistema permite gestionar la instalación, actualización y eliminación de software de forma automatizada.

Los paquetes RPM incluyen:

Archivos del programa.

Información sobre dependencias.

Scripts de instalación y desinstalación.

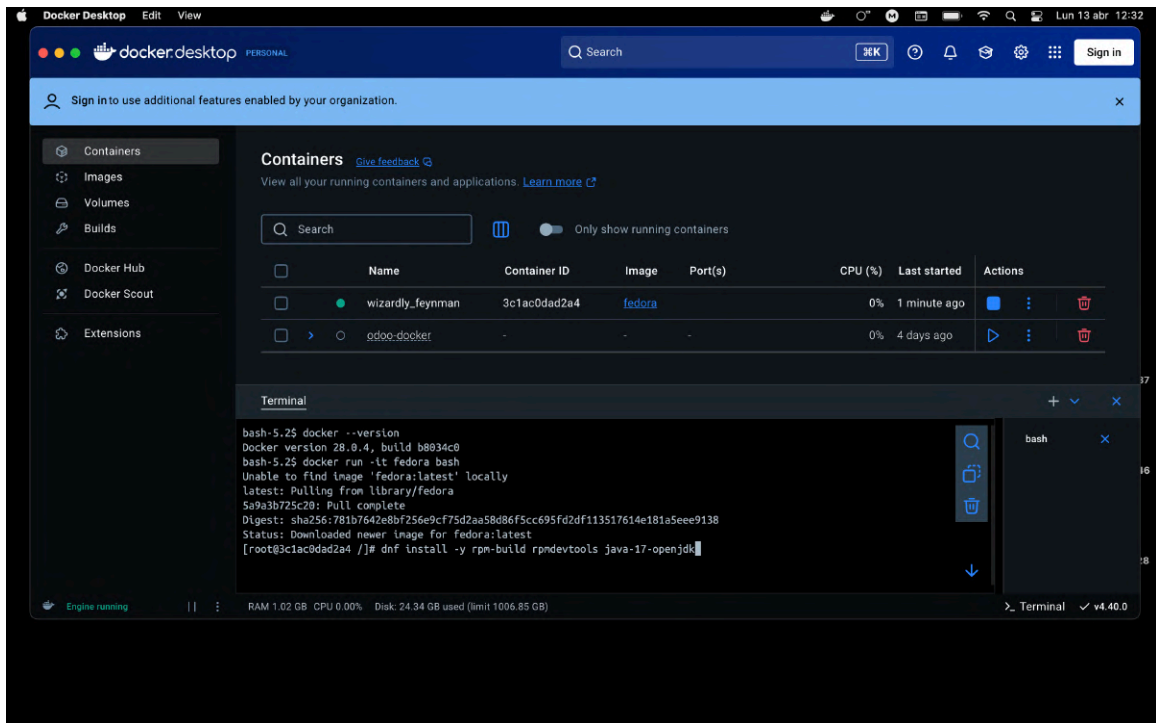
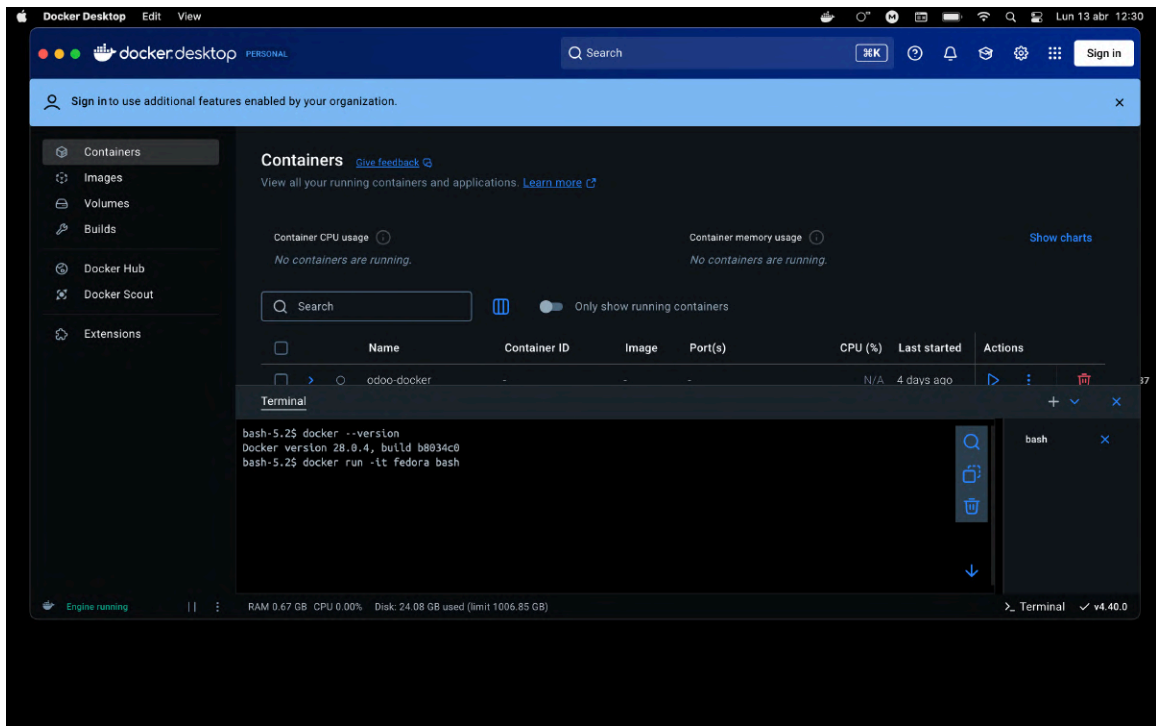
Una de sus principales ventajas es que permite garantizar la integridad del software y simplificar su distribución, especialmente en entornos profesionales.

2. Preparación del entorno

Se ha utilizado Docker Desktop para ejecutar un contenedor Fedora.

Comando utilizado:

```
docker run -it --name rpm-practica fedora bash
```



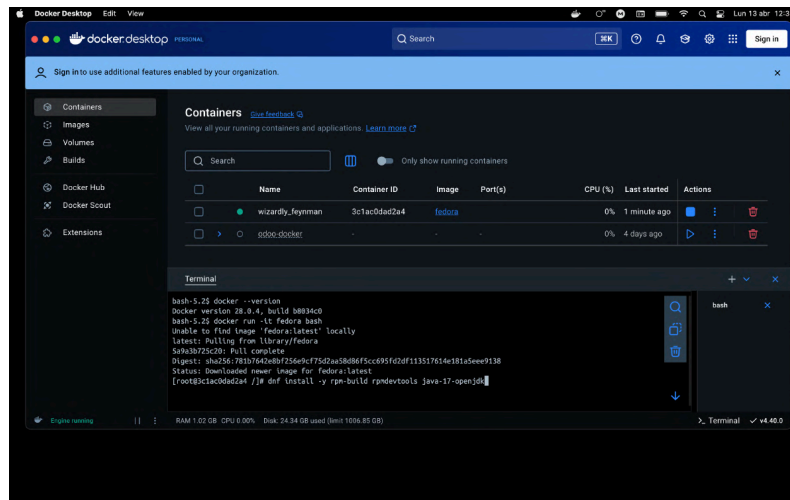
3. Instalación de herramientas

Instalación de herramientas necesarias para la creación de RPM y Java:

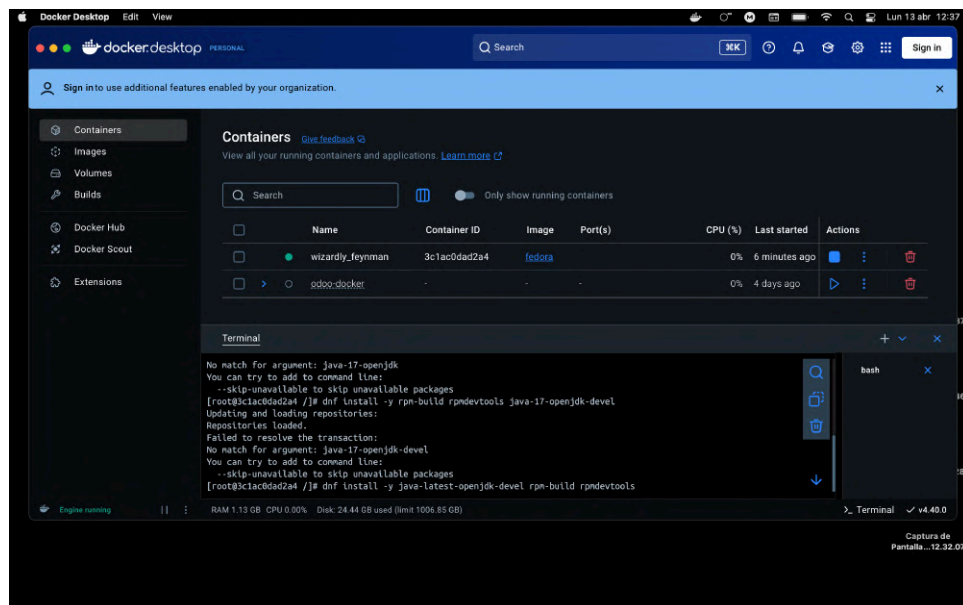
Primero hice `dnf install -y rpm-build rpmdevtools java-17-openjdk` y como me dio error aquí muestro en las capturas tuve que hacer :

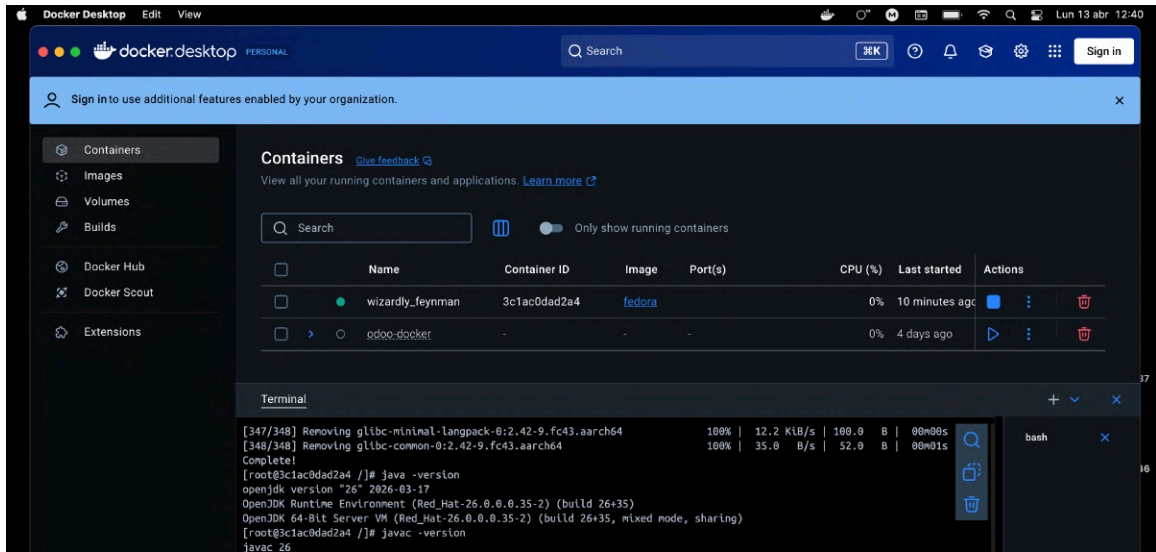
`dnf install -y rpm-build rpmdevtools java-latest-openjdk-devel`

Asi si me lo instalo bien y compruebo que tanto java como el jdk están instalados correctamente, muestro aquí capturas de todo el proceso :



En las siguientes capturas se ve como el primer comando da error y el segundo ya tiene la versión de java y javac correcta instalada :

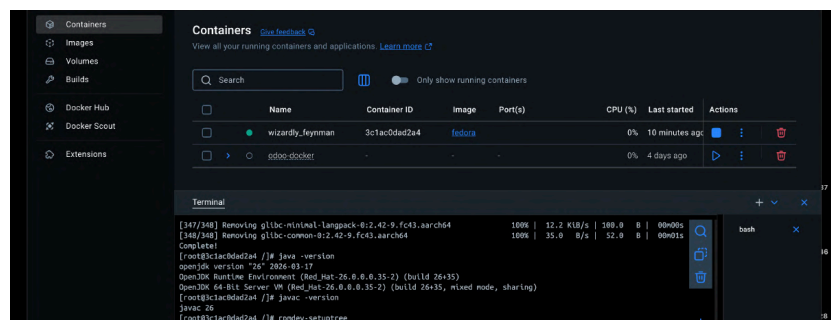




4. Preparación del entorno RPM

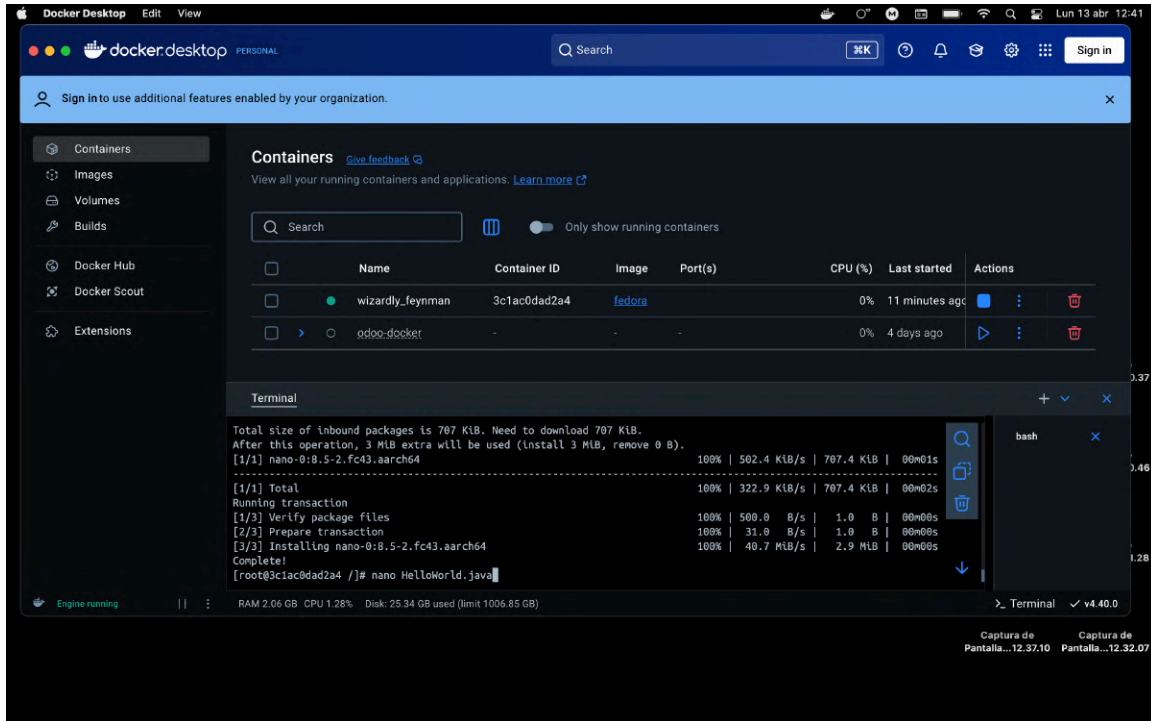
Se inicializa la estructura de directorios para rpmbuild:

rpmdev-setuptree



5. Desarrollo de la aplicación Java

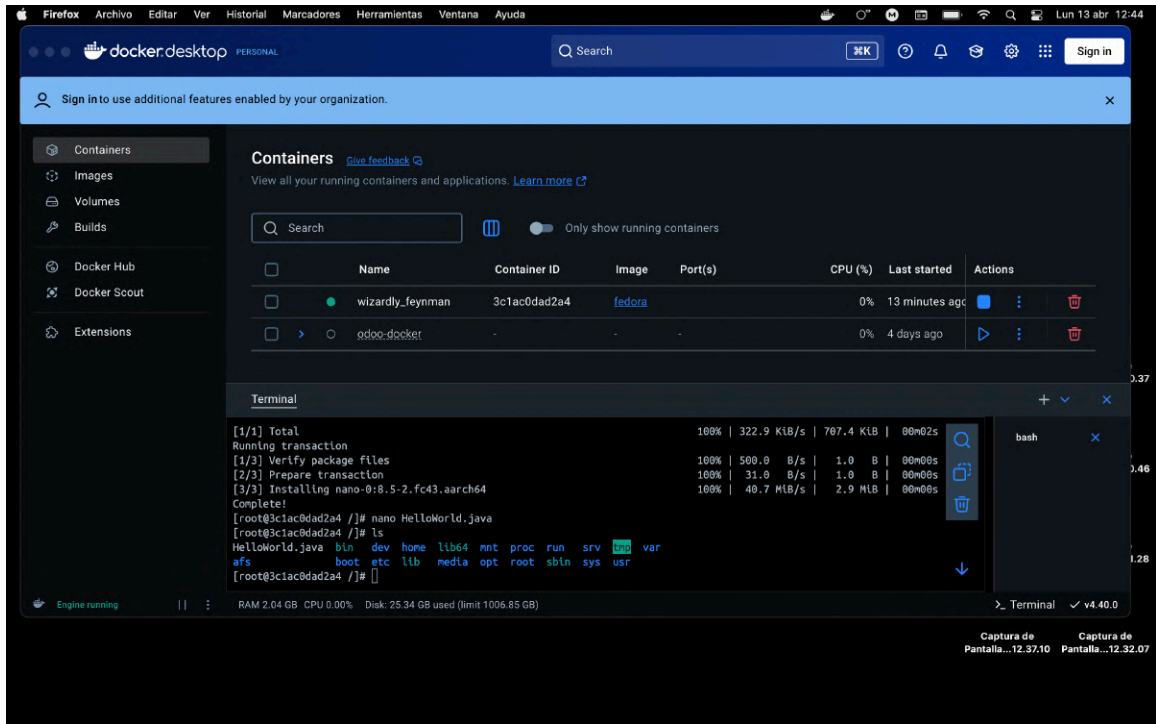
Se crea una aplicación Java sencilla. Instalamos nano para editar el archivo HelloWorld.java que crearemos , le ponemos un código básico con basico Hello world .



Codigo Helloworld.java :

```
public class HelloWorld {  
  
    public static void main(String[] args) {  
  
        System.out.println("Hola desde RPM creado en Docker con Fedora");  
  
    }  
  
}
```

Comprobamos con el comando **ls** que esta en directorio el archivo :



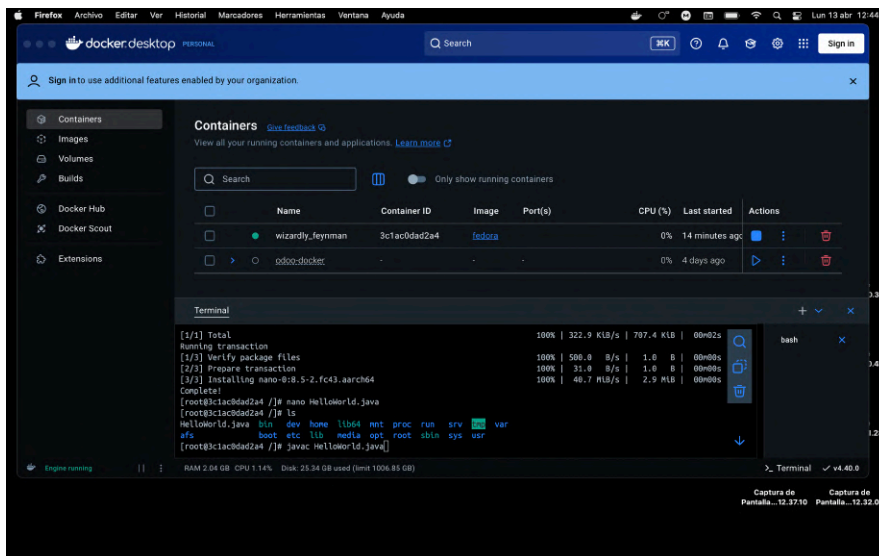
6. Compilación y creación del JAR

Compilación:

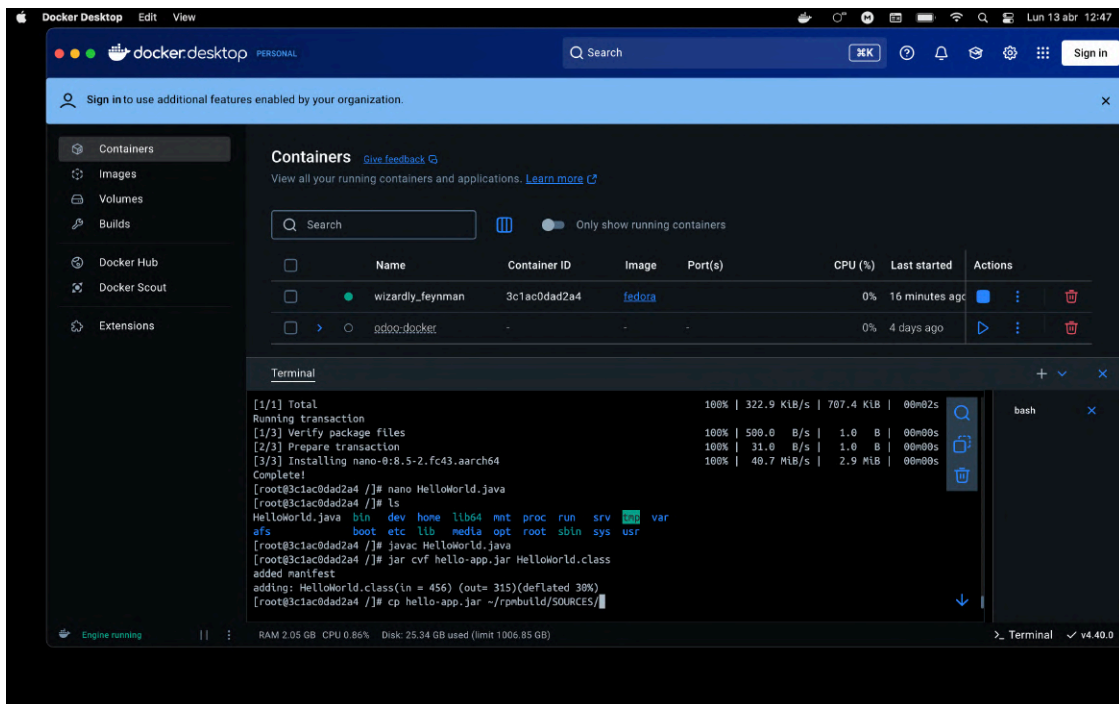
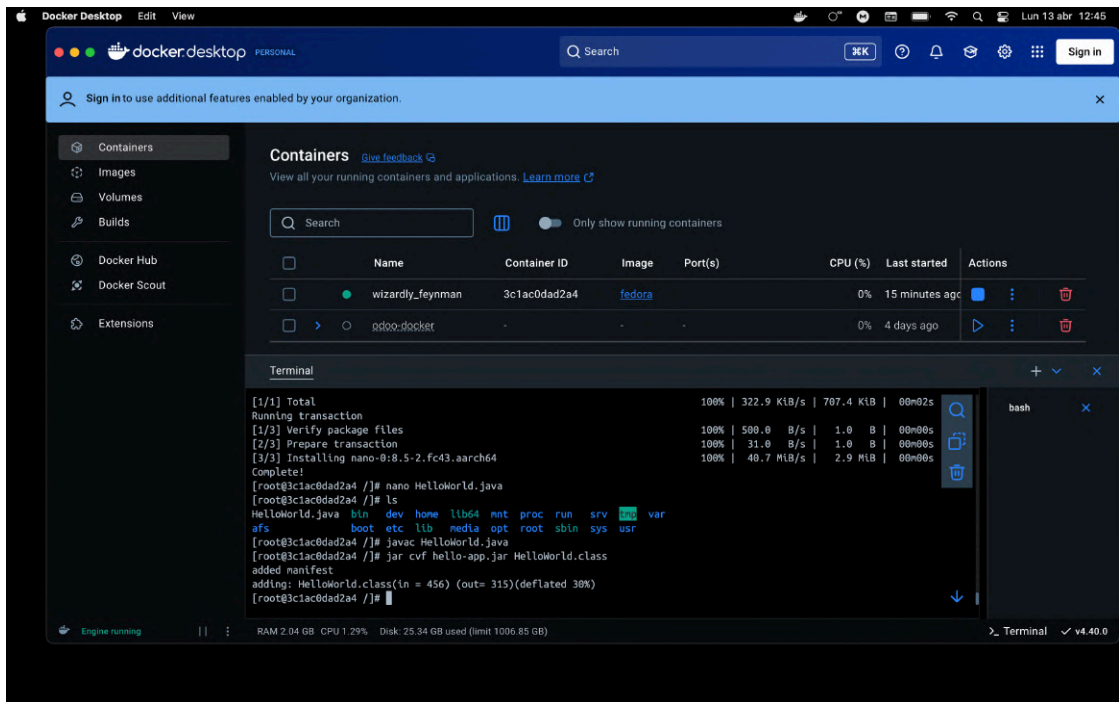
```
javac HelloWorld.java
```

Creación del JAR:

```
jar cvf hello-app.jar HelloWorld.class
```



SS

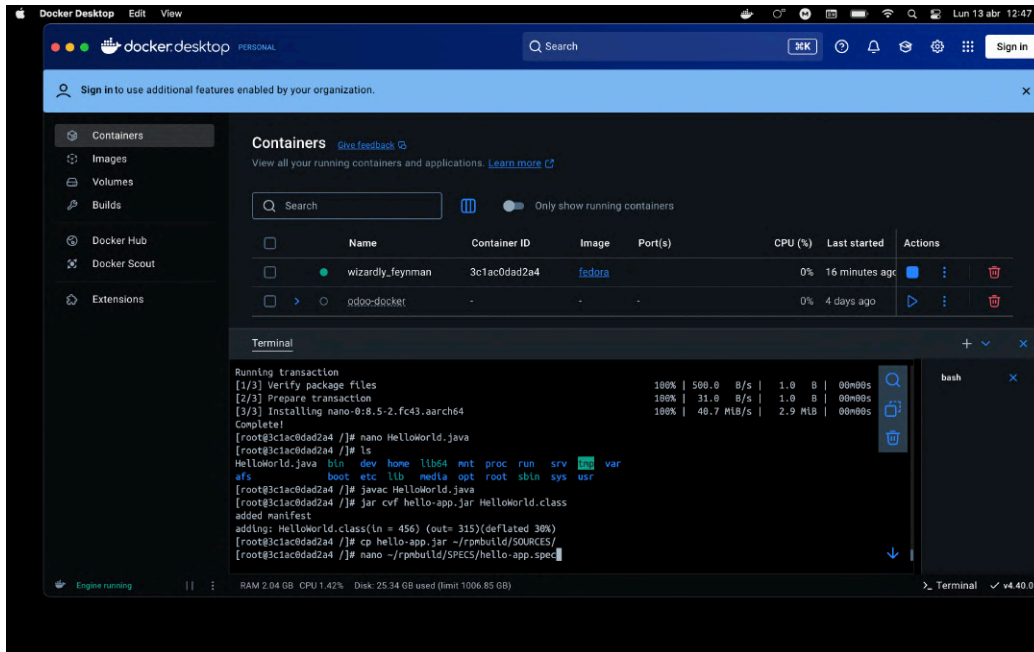


7. Creación del paquete RPM

Se crea el archivo SPEC y se construye el paquete RPM.

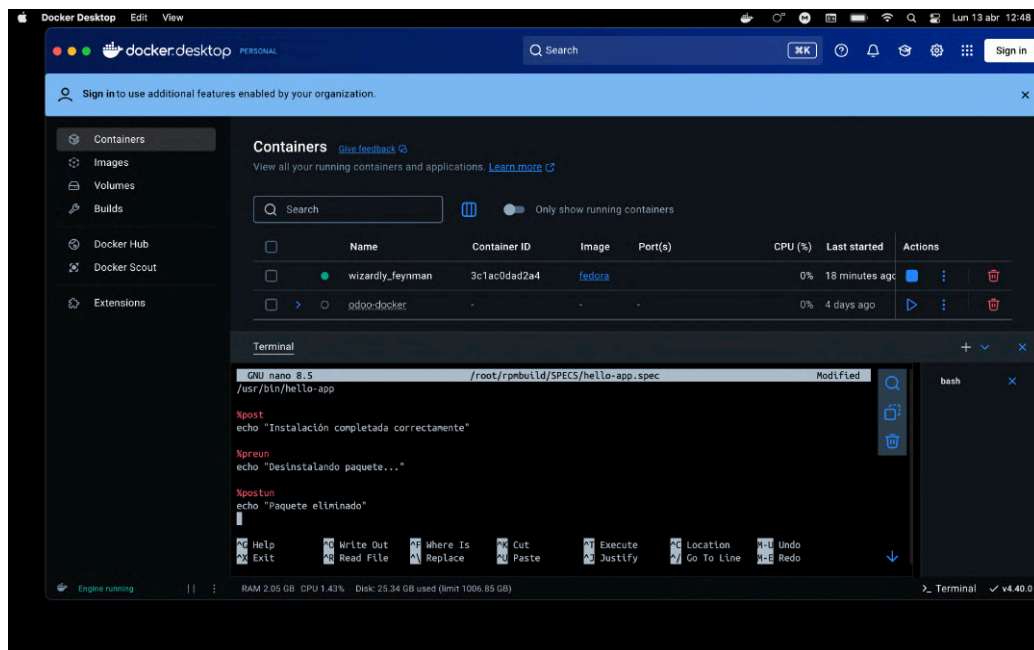
```
rpmbuild -ba ~/rpmbuild/SPECS/hello-app.spec
```

Creamos primero con nano el archive spec con el código necesario :



The screenshot shows the Docker Desktop interface. On the left is a sidebar with navigation options: Containers, Images, Volumes, Builds, Docker Hub, Docker Scout, and Extensions. The main panel is titled 'Containers' and shows a table of running containers. Below this is a terminal window. The terminal output shows the execution of the `rpmbuild -ba` command, which successfully builds the RPM package. The terminal text is as follows:

```
Running transaction
[1/3] Verify package files
[2/3] Prepare transaction
[3/3] Installing nano-0:8.5-2.fc43.aarch64
Complete!
[root@3c1ac0dad2a4] /# nano HelloWorld.java
[root@3c1ac0dad2a4] /# ls
HelloWorld.java bin dev home lib64 mnt proc run srv var
afs boot etc lib media opt root sbin sys usr
[root@3c1ac0dad2a4] /# javac HelloWorld.java
[root@3c1ac0dad2a4] /# jar cvf hello-app.jar HelloWorld.class
added manifest
adding: HelloWorld.class(in = 456)(out= 315)(deflated 30%)
[root@3c1ac0dad2a4] /# cp hello-app.jar ~/rpmbuild/SOURCES/
[root@3c1ac0dad2a4] /# nano ~/rpmbuild/SPECS/hello-app.spec
```



The screenshot shows the Docker Desktop interface with the terminal window open. The terminal is now running the `nano` editor to edit the `~/rpmbuild/SPECS/hello-app.spec` file. The editor shows the following content:

```
GNU nano 8.5 /root/rpmbuild/SPECS/hello-app.spec Modified
/usr/bin/hello-app

%post
echo "Instalación completada correctamente"

%preun
echo "Desinstalando paquete..."

%postun
echo "Paquete eliminado"
```

Codigo del archive hello-app.spec

Name: hello-app

Version: 1.0

Release: 1%{?dist}

Summary: Aplicación Java empaquetada en RPM

License: MIT

Source0: hello-app.jar

BuildArch: noarch

Requires: java-latest-openjdk

%description

Aplicación Java simple empaquetada en RPM usando Docker.

%prep

%build

%install

mkdir -p %{buildroot}/opt/hello-app

cp %{SOURCE0} %{buildroot}/opt/hello-app/

mkdir -p %{buildroot}/usr/bin

echo '#!/bin/bash

java -jar /opt/hello-app/hello-app.jar' > %{buildroot}/usr/bin/hello-app

chmod +x %{buildroot}/usr/bin/hello-app

%files

/opt/hello-app/hello-app.jar

/usr/bin/hello-app

%post

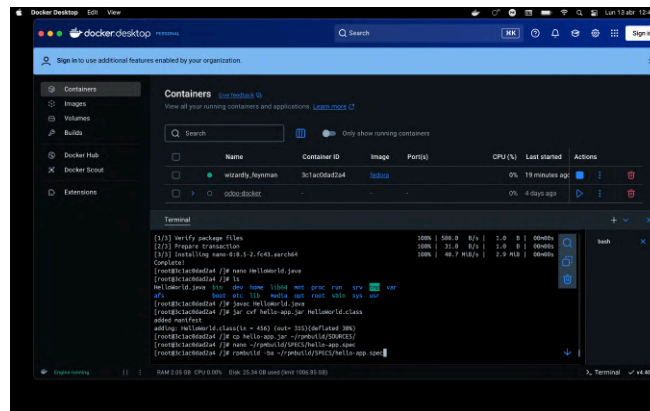
echo "Instalación completada correctamente"

%preun

echo "Desinstalando paquete..."

%postun

echo "Paquete eliminado"



El archivo SPEC es el elemento más importante en la creación de un paquete RPM, ya que define cómo se construye, instala y gestiona el software.

Sus secciones principales son:

Name, Version, Release: identifican el paquete.

Summary: breve descripción.

License: tipo de licencia del software.

Source0: archivo fuente (en este caso el JAR).

Requires: dependencias necesarias para ejecutar el programa.

Secciones de ejecución:

%prep: preparación del entorno (no usado en este caso).

%build: compilación (no necesaria al usar un JAR ya compilado).

%install: copia de archivos al sistema.

%files: lista de archivos que incluirá el paquete.

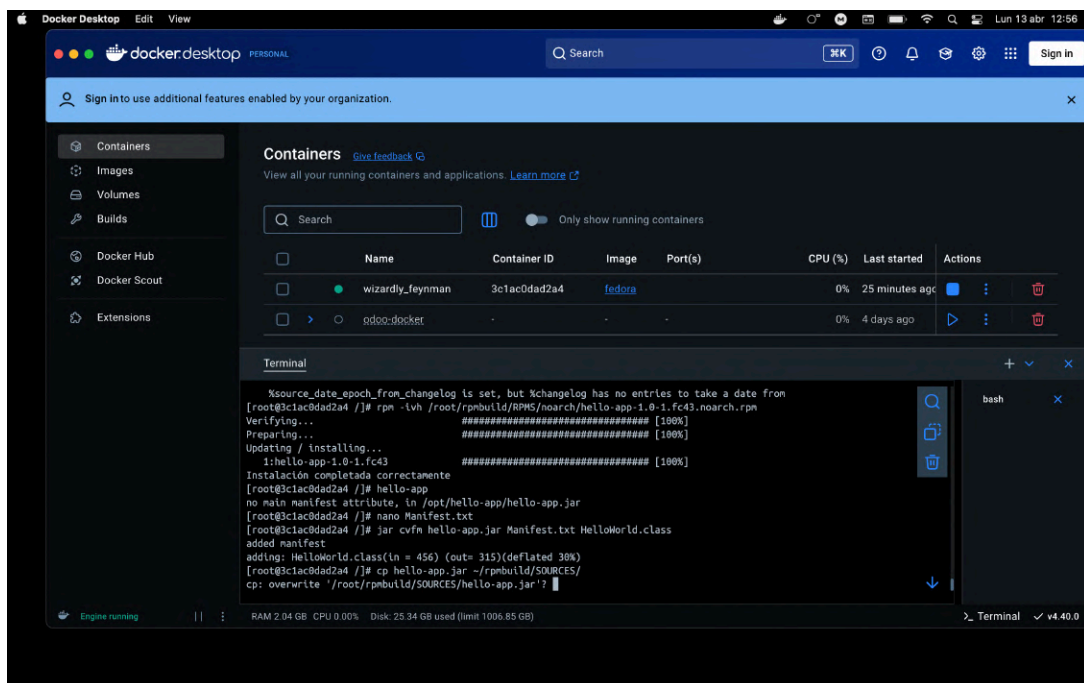
Scripts:

%post: se ejecuta tras la instalación.

%preun y %postun: se ejecutan durante la desinstalación.

Este archivo permite automatizar completamente la instalación del software.

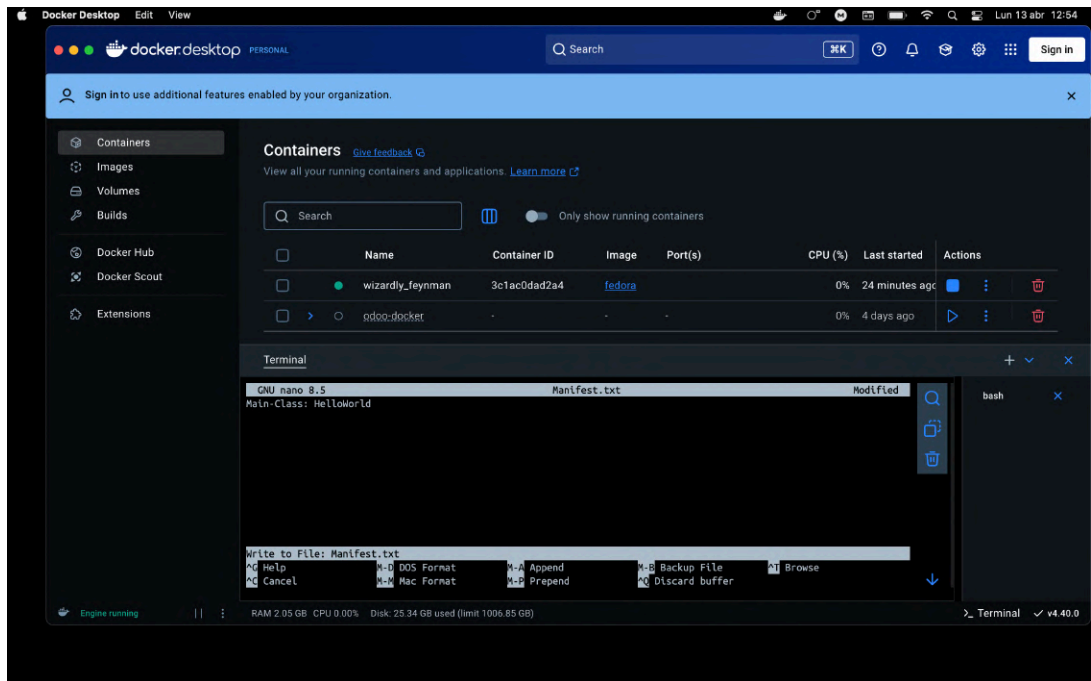
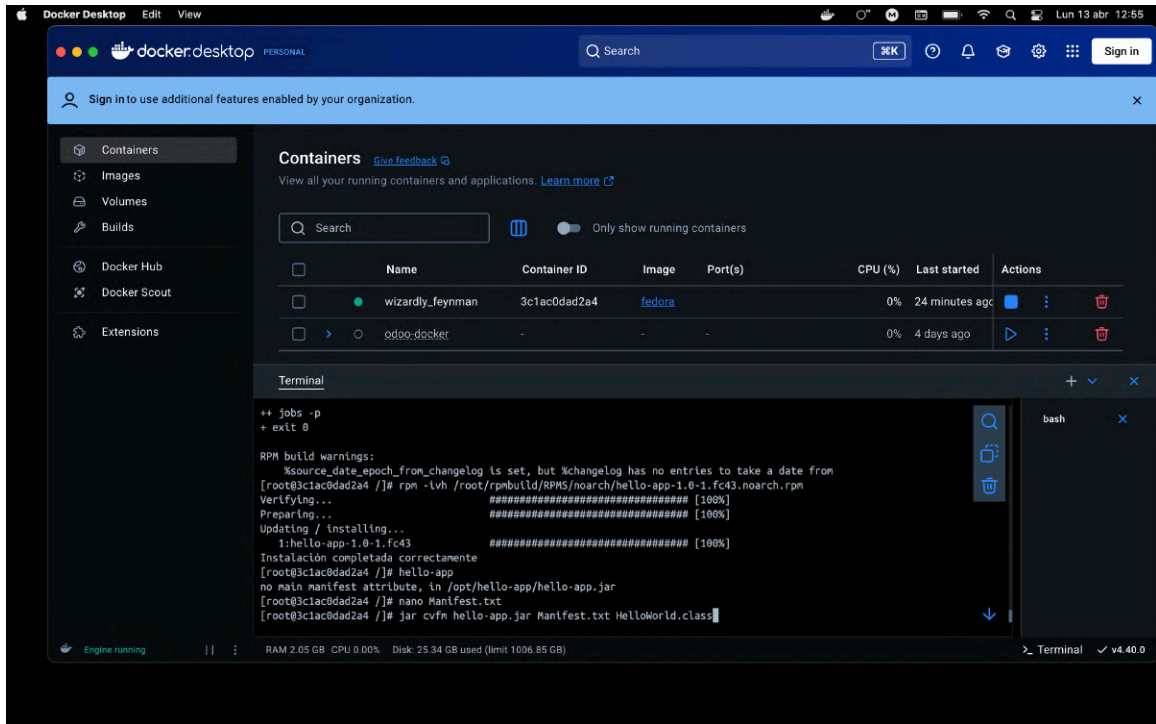
Encontramos un pequeño problema en el proceso , y creamos el archivo Manifest.txt para que sepa cual es la clase principal de ejecución por que al instalar el rpm y ejecutar el HelloWorld.java me dice que no lo tiene y tenemos que volver a hacer todo el proceso del empaquetado de nuevo por ese error , aquí muestro las capturas de pantallas con el error y la solución al problema :

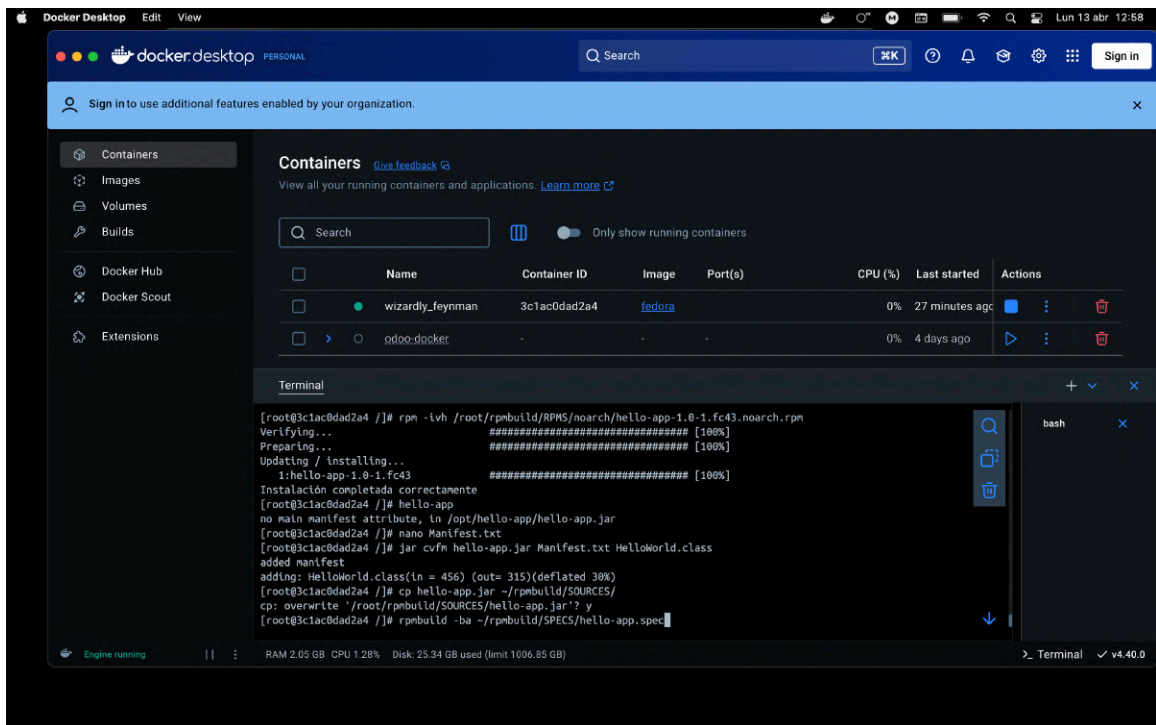
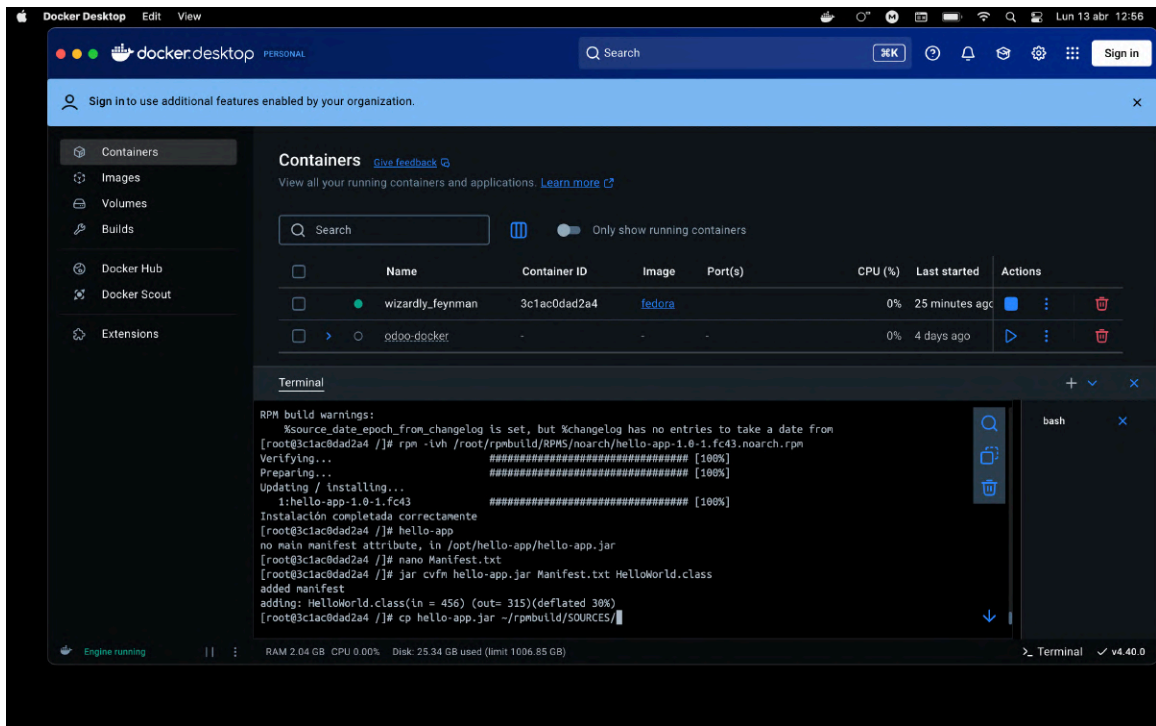


The screenshot shows the Docker Desktop application interface. On the left is a sidebar with navigation options: Containers, Images, Volumes, Builds, Docker Hub, Docker Scout, and Extensions. The main area is titled 'Containers' and displays a table of running containers. Below the table is a terminal window showing the output of RPM commands.

Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
wizardly_feynman	3c1ac0dad2a4	fedora		0%	25 minutes ago	[Stop] [Restart] [Delete]
cdoo:docker	-	-	-	0%	4 days ago	[Stop] [Restart] [Delete]

```
%source_date_epoch_from_changelog is set, but %changelog has no entries to take a date from
[root@3c1ac0dad2a4 /]# rpm -ivh /root/rpmbuild/RPMS/noarch/hello-app-1.0-1.fc43.noarch.rpm
Verifying...
Preparing...
Updating / Installing...
1:hello-app-1.0-1.fc43
Instalación completada correctamente
[root@3c1ac0dad2a4 /]# hello-app
no main manifest attribute, in /opt/hello-app/hello-app.jar
[root@3c1ac0dad2a4 /]# nano Manifest.txt
[root@3c1ac0dad2a4 /]# jar cvfm hello-app.jar Manifest.txt HelloWorld.class
added manifest
adding: HelloWorld.class(in = 456) (out= 315)(deflated 38%)
[root@3c1ac0dad2a4 /]# cp hello-app.jar ~/rpmbuild/SOURCES/
cp: overwrite '/root/rpmbuild/SOURCES/hello-app.jar'?
```

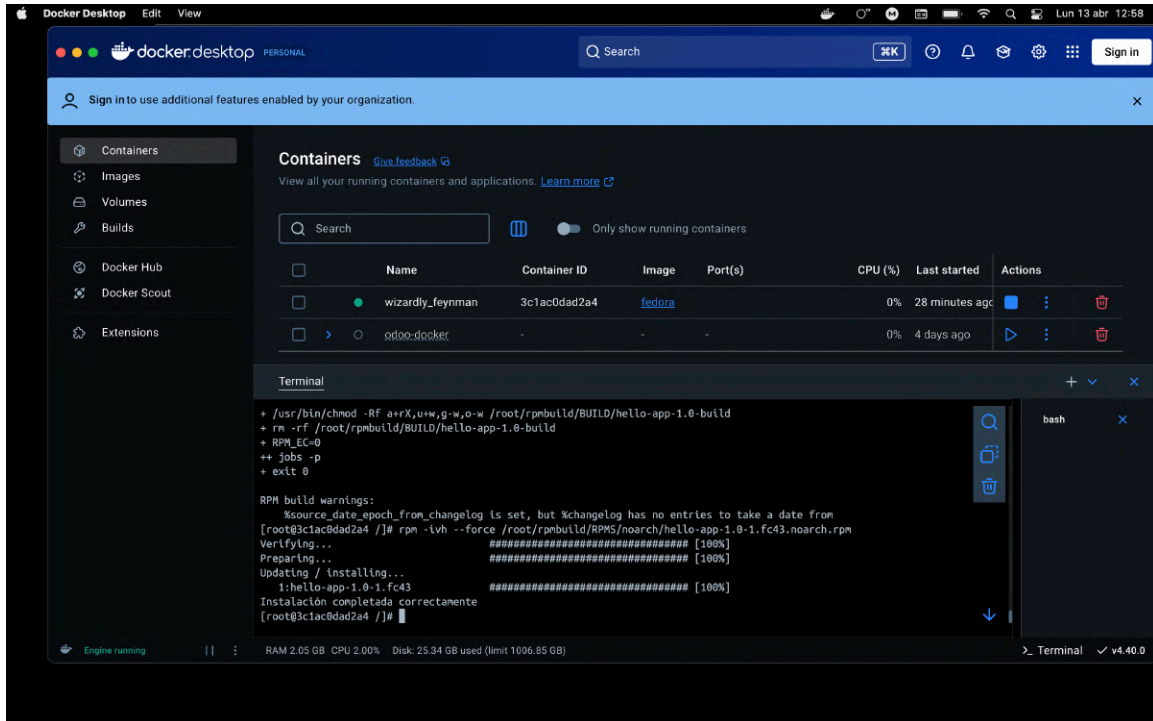




8. Instalación del RPM

Instalación del paquete generado:

```
rpm -ivh hello-app-1.0-1.fc43.noarch.rpm
```



Verificación de la instalación

Para comprobar que el paquete se ha instalado correctamente, se pueden usar los siguientes comandos:

```
rpm -ql hello-app
```

Este comando lista todos los archivos instalados por el paquete.

También se puede comprobar si está instalado:

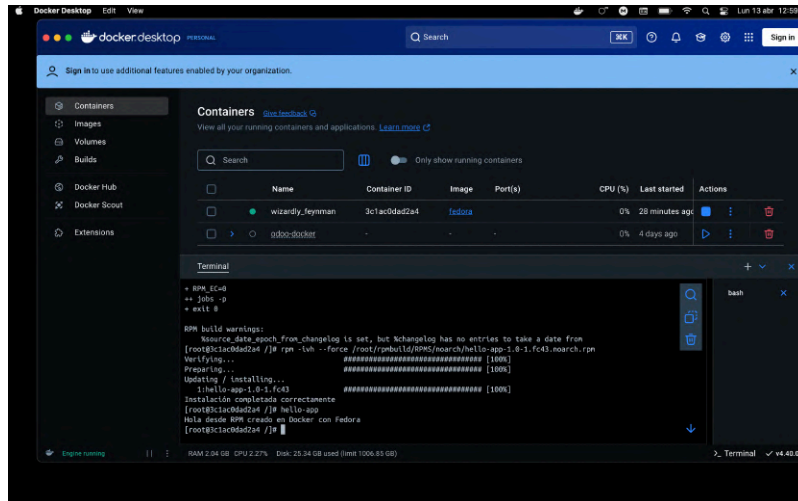
```
rpm -q hello-app
```

9. Ejecución y pruebas

Ejecución del programa:

hello-app

Resultado esperado: Hola desde RPM creado en Docker con Fedora



Desinstalación del paquete

Una de las ventajas de los paquetes RPM es que permiten eliminar el software de forma limpia:

`rpm -e hello-app`

Este comando elimina todos los archivos instalados y ejecuta los scripts definidos en el archivo SPEC.

Esto garantiza que no queden residuos en el sistema.

10. Gestión de dependencias en JAR y firma digital

10.1 Diferencias entre incluir o no las librerías de dependencias en un JAR

A la hora de distribuir una aplicación Java en formato JAR, existen dos enfoques principales: incluir todas las dependencias dentro del JAR (JAR “fat” o “uber”) o distribuir el JAR sin dependencias externas.

JAR con dependencias incluidas (Fat JAR)

Consiste en empaquetar en un único archivo JAR tanto la aplicación como todas las librerías necesarias para su ejecución.

Ventajas:

Facilita la distribución, ya que todo está contenido en un solo archivo.

Evita problemas de compatibilidad de versiones en el sistema del usuario.

No requiere instalación previa de librerías adicionales.

Desventajas:

El tamaño del archivo es mayor.

Puede haber duplicación de librerías si varias aplicaciones incluyen las mismas.

Menor flexibilidad para actualizar dependencias de forma independiente.

Ejemplo: Un proyecto que utiliza una librería como log4j o spring-core puede empaquetarse completamente en un único JAR ejecutable usando herramientas como Maven (maven-shade-plugin).

JAR sin dependencias (Thin JAR)

En este caso, el JAR contiene únicamente el código de la aplicación, y las dependencias deben estar instaladas previamente en el sistema o gestionarse externamente.

Ventajas:

Menor tamaño del archivo.

Permite reutilizar librerías ya instaladas en el sistema.

Facilita la actualización independiente de las dependencias.

Desventajas:

Mayor complejidad en la instalación.

Riesgo de incompatibilidades de versiones.

Requiere configurar correctamente el classpath.

Ejemplo: Una aplicación Java que depende de una librería externa como mysql-connector necesitará que dicha librería esté presente en el sistema y correctamente referenciada al ejecutar:

```
java -cp miapp.jar:mysql-connector.jar com.ejemplo.Main
```

10.2 Ejemplo práctico de uso de un JAR sin dependencias

Un caso común de uso de JAR sin dependencias se da en entornos empresariales donde múltiples aplicaciones comparten librerías comunes.

Por ejemplo, en un servidor Linux donde varias aplicaciones Java utilizan la misma versión de una librería, estas pueden almacenarse en un directorio común (por ejemplo /opt/libs). De este modo:

Se reduce el espacio en disco.

Se simplifica el mantenimiento de versiones.

Se mejora la eficiencia del sistema.

En este contexto, el JAR principal de la aplicación se distribuye sin dependencias, y se ejecuta configurando el classpath para incluir dichas librerías compartidas.

10.3 Proceso para añadir un certificado digital a un JAR

La firma digital de un JAR garantiza la integridad del archivo y la autenticidad del desarrollador. Es especialmente importante en aplicaciones distribuidas públicamente o en entornos donde la seguridad es crítica.

◆ Paso 1: Crear un keystore

Se genera un almacén de claves que contendrá el certificado:

```
keytool -genkeypair -alias mi-certificado -keyalg RSA -keystore mi-keystore.jks
```

Durante este proceso se solicitarán datos como nombre, organización y contraseña.

◆ Paso 2: Firmar el JAR con jarsigner

Una vez creado el keystore, se firma el archivo JAR:

```
jarsigner -keystore mi-keystore.jks hello-app.jar mi-certificado
```

Esto añade una firma digital al JAR que permite verificar que no ha sido modificado.

◆ Paso 3: Verificar la firma

Para comprobar que el JAR está correctamente firmado:

```
jarsigner -verify hello-app.jar
```

Si el proceso es correcto, se indicará que la firma es válida.

◆ Importancia de la firma digital

Garantiza que el archivo no ha sido alterado desde su firma.

Permite identificar al autor del software.

Es necesaria en ciertos entornos, como aplicaciones distribuidas por internet o sistemas con políticas de seguridad estrictas.

10.4 Conclusión sobre la gestión de dependencias en JAR y firma digital

La gestión de dependencias y la firma digital son aspectos clave en la distribución de aplicaciones Java. Mientras que el uso de JAR con dependencias facilita la instalación, los JAR sin dependencias ofrecen mayor flexibilidad en entornos controlados. Por otro lado, la firma digital añade una capa esencial de seguridad que asegura la integridad y autenticidad del software distribuido. He analizado también la gestión de dependencias y la firma digital puesto que había dos criterios de calificación de la tarea y he preferido cumplir ambos y así se detalla mas el tema del empaquetado rpm .

11. Conclusión tarea

Se ha completado con éxito el proceso de empaquetado de una aplicación Java en formato RPM, utilizando herramientas estándar de Linux dentro de un contenedor Docker. El resultado final es un paquete instalable y ejecutable de forma automatizada.

12. Bibliografia

Unidad 7 Interfaces

Documentacion rpm :

<https://rpm.org/docs/6.0.x/api/index.html>

Documentacion Fedora :

<https://docs.fedoraproject.org/en-US/packaging-guidelines/>